MACHINE LEARNING IN PHYSICS
FOUNDATIONS 3

HARRISON B. PROSPER

PHY6937 / PHY4936

Recap

What have we learned so far?

1. Machine learning models are trained by minimizing the empirical risk (aka average loss):

$$R(\omega) = \frac{1}{N}\sum_{i=1}^{N} L(y_i, f_i)$$

2. The loss function, $L(y, f)$, is chosen according to the task at hand.

Recap

3. The empirical risk is an *unbiased* Monte Carlo estimate of the risk functional

$$R[f] = \int dx \int dy\, L(y, f)\, p(x, y)$$

where $p(x, y)dxdy$ is the probability of the data.

4. Given f, with inputs x and targets y, and the loss $L(y, f)$, minimizing the risk functional yields the important result

$$\int dy\, \frac{\partial L}{\partial f} p(y \mid x) = 0 \rightarrow \mathbb{E}_{p(y|x)} \left[\frac{\partial L}{\partial f}\right] = 0$$

COMMON LOSS FUNCTIONS

Common Loss Functions (1)

Quadratic loss (often used for regression)

$$L(y, f) = (y - f)^2$$

$$y \in \mathbb{R}$$

Binary cross entropy (often used for classification)

$$L(y, f) = -[y \log f + (1 - y) \log(1 - f)]$$

$$y \in \{0, 1\}$$

Quadratic Loss

Quadratic loss: $L(y, f) = (y - f)^2$

$$\mathbb{E}_{p(y|x)}\left[\frac{\partial L}{\partial f}\right] = 0$$

Solution

$$f_{\omega^*}(x) = \mathbb{E}_{p(y|x)}[y]$$

As noted, in the previous lecture, given a sufficiently flexible model, this result depends solely on the form of the loss function and the conditional probability, $p(y|x)dy$, of the data.

Binary Cross Entropy Loss (1)

Binary cross entropy loss : $-[y \log f + (1-y)\log(1-f)]$

$$\mathbb{E}_{p(y|x)}\left[\frac{\partial L}{\partial f}\right] = 0$$

$$\mathbb{E}_{p(y|x)}\left[\frac{y}{f} - \frac{(1-y)}{1-f}\right] = 0$$

Therefore, for y in the unit interval

$$f_{\omega^*}(x) = \mathbb{E}_{p(y|x)}[y]$$

But when $y = \{0, 1\}$ this yields

$$f_{\omega^*}(x) = p(1 \mid x)$$

Other Loss Functions

Exponential loss

$$L(y, f) = \exp\left(-\frac{wyf}{2}\right), \quad y \in \{-1, +1\}$$

Quantile loss $(0 \leq \tau \leq 1)$

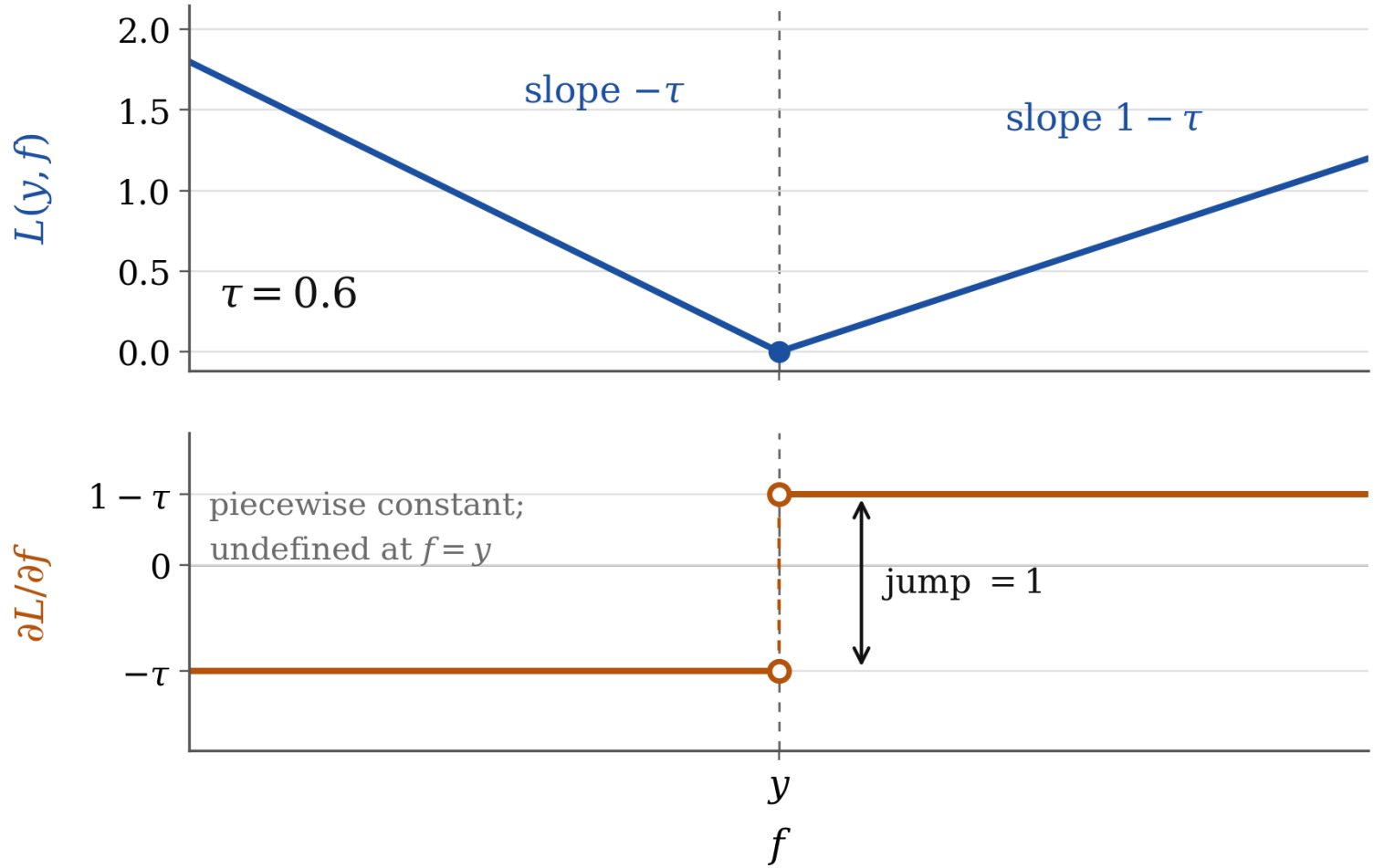
$$L(y, f) = \begin{cases} \tau(y - f) & y \geq f \\ (1 - \tau)(f - y) & y < f \end{cases}$$

Zero-One loss

$$L(y, f) = \begin{cases} 0 & |y - f| \leq b \\ 1 & |y - f| > b \end{cases}$$

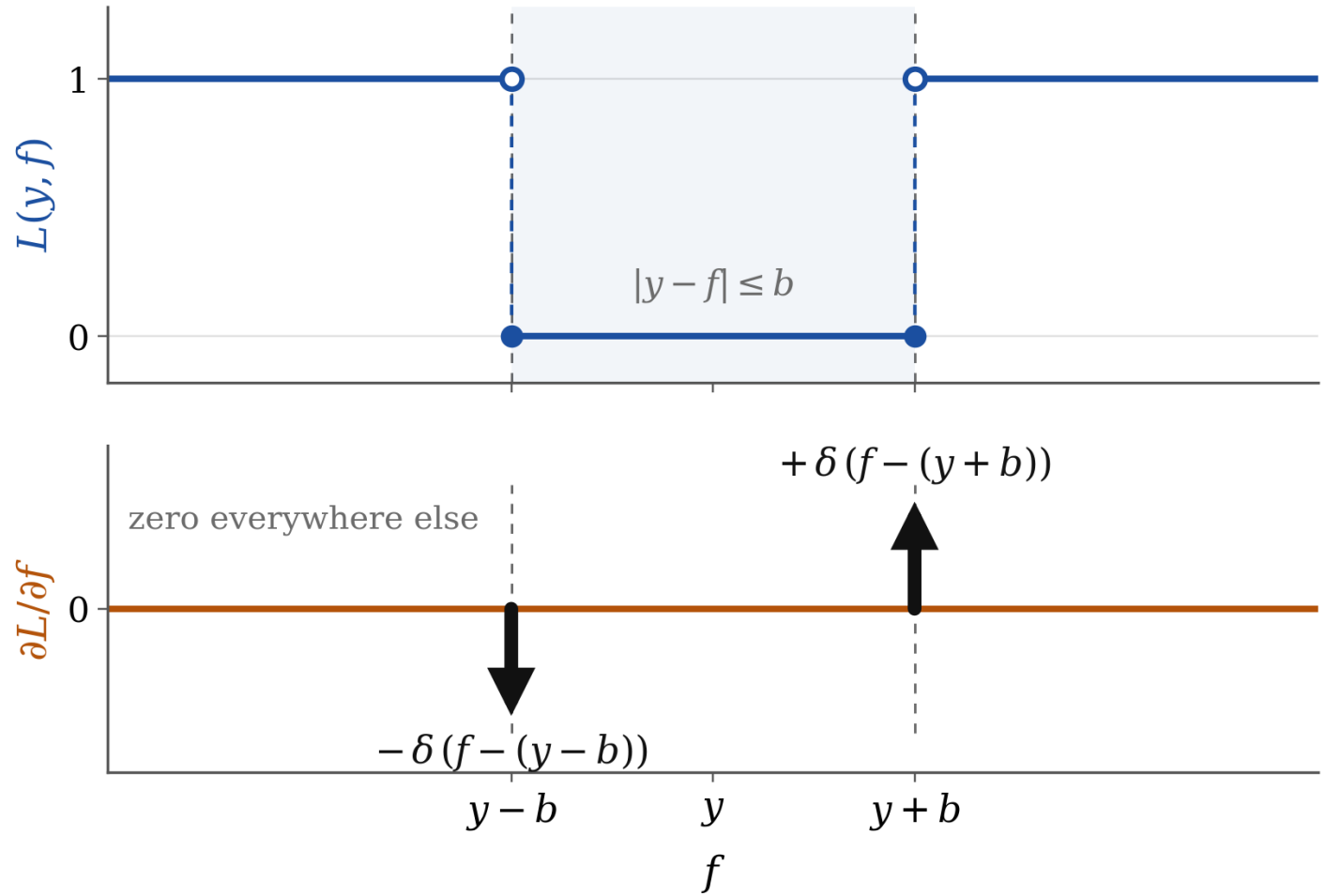
Quantile loss ($0 \le \tau \le 1$)

$$L(y,f) = \begin{cases} \tau(y-f) & y \ge f \\ (1-\tau)(f-y) & y < f \end{cases}$$



Zero-One loss

$$L(y,f) = \begin{cases} 0 & |y-f| \le b \\ 1 & |y-f| > b \end{cases}$$



BINARY CLASSIFIERS

Binary Classifiers (1)

Starting with

$$\int dy \frac{\partial L}{\partial f} p(y|x) = 0$$

we've shown that a binary classifier $f_\omega(x)$ trained using the cross-entropy loss

$$L(y, f) = -[y \log f + (1 - y) \log(1 - f)]$$

approximates the **class probability** $p(1|x)$, that is, the probability that an object characterized by the **features** x belongs to the class labeled $y = 1$.

Binary Classifiers (2)

From Bayes' theorem, we can write the class probability as

$$p(1|x) = \frac{p(x|1)\,\pi(1)}{p(x)}$$

where $p(x) = p(x|1)\,\pi(1) + p(x|0)\,\pi(0)$ and

- $p(x|k)$ is the probability density of x for class $y = k$,
- $\pi(k)$ is the **prior probability** of class $y = k$, and
- $\pi(1) + \pi(0) = 1.$

Binary Classifiers (3)

When the training dataset is **balanced** that is, $\pi(1) = \pi(0)$, the class probability $p(1|x)$ becomes the **discriminant**,

$$D(x) = \frac{p(x|1)}{p(x|1) + p(x|0)}$$

This is one of the most useful results in scientific machine learning.

Binary Classifiers (4)

It is possible to train a binary classifier using an **unbalanced** dataset: $\pi(1) \neq \pi(0)$.

But if $\pi(1) \ll \pi(0)$ why might this be very challenging?

Binary Classifiers (4)

Let's start with Bayes' theorem

$$p(1|x) = \frac{p(x|1)\pi(1)}{p(x|1)\pi(1) + p(x|0)\pi(0)}$$

Dividing the numerator and denominator by $p(x|1) + p(x|0)$ yields

$$p(1|x) = \frac{D(x)\pi(1)}{D(x)\pi(1) + [1 - D(x)]\pi(0)}$$

This shows that, $p(1|x)$ can be computed from a model trained with a *balanced* dataset. There is no need to train with the actual class prior probabilities, as is sometimes asserted.

CLASSIFIER PERFORMANCE

Classifier Performance (1)

Consider a classifier, $D(x)$, the indicator function $I[*]$, the targets y, and a threshold t on $D(x)$ above which one is prepared to assign x to the "positive" class and below which one assigns it to the "negative" class.

The following counts can be computed:

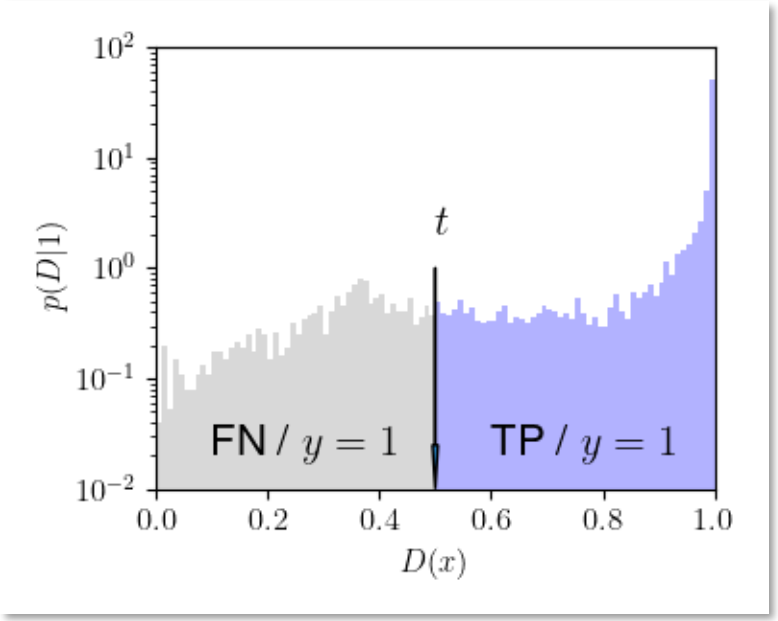
1. False Negatives (FN): $\sum_{i,\, \boldsymbol{y_i=1}} I[D(x_i) \leq t]$
2. True Positives (TP): $\sum_{i,\, \boldsymbol{y_i=1}} I[D(x_i) > t]$
3. True Negatives (TN): $\sum_{i,\boldsymbol{y_i=0}} I[D(x_i) \leq t]$
4. False Positives (FP): $\sum_{\boldsymbol{i},\, \boldsymbol{y_i=0}} I[D(x_i) > t]$

$I[z] = 1$ if z is true, 0 otherwise.

Classifier Performance (2)

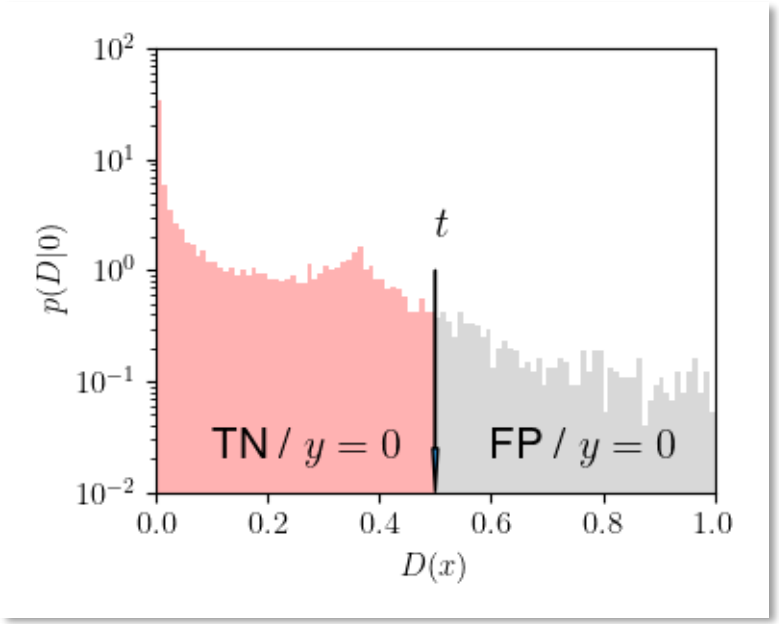
FN: $\sum_{i,\, y_i=1} I[D(x_i) \leq t]$ $t = 0.5$

TP: $\sum_{i,\, y_i=1} I[D(x_i) > t]$



$y = 1$

$y = 0$



TN: $\sum_{i,\, y_i=0} I[D(x_i) \leq t]$

FP: $\sum_{i,\, y_i=0} I[D(x_i) > t]$

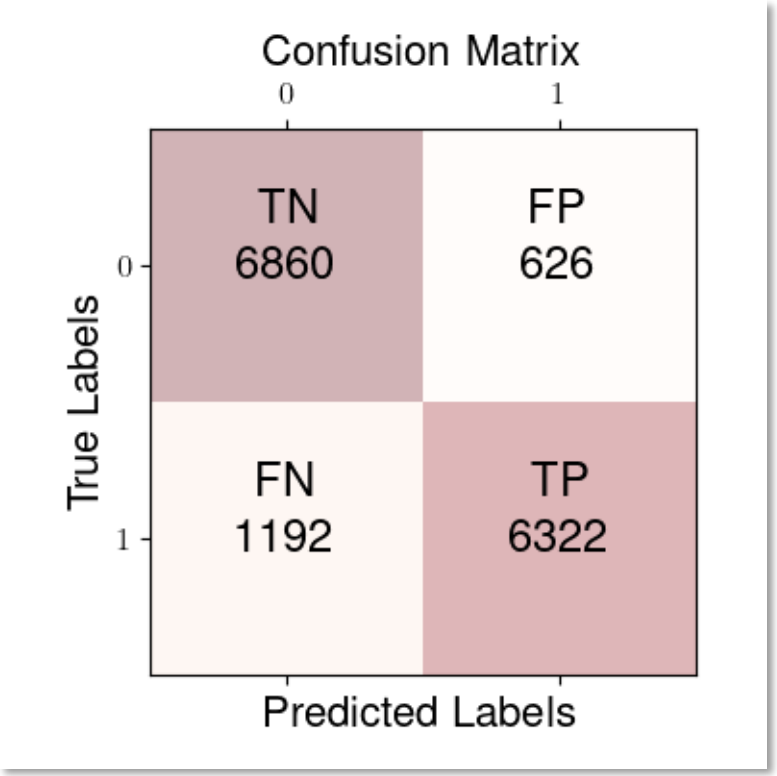
Confusion Matrix

FN: $\sum_{i,\, y_i=1} I[D(x_i) \leq t]$

TP: $\sum_{i,\, y_i=1} I[D(x_i) > t]$

The confusion matrix is a simple way to summarize the counts in the four disjoint regions.

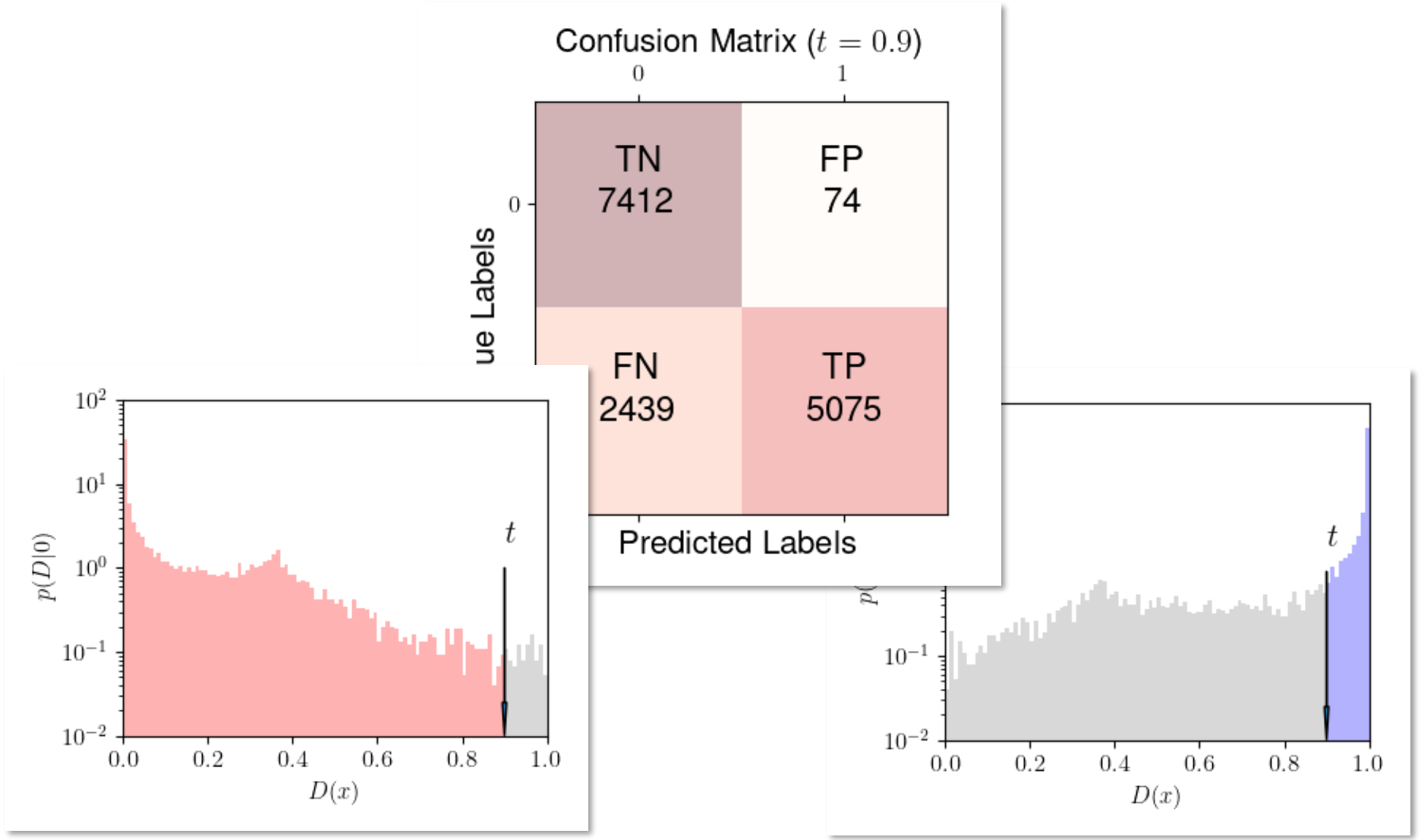
A perfectly separable dataset would yield a diagonal matrix.



TN: $\sum_{i,\, y_i=0} I[D(x_i) \leq t]$

FP: $\sum_{i,\, y_i=0} I[D(x_i) > t]$

Confusion Matrix ($t = 0.9$)



Accuracy, Precision, Recall

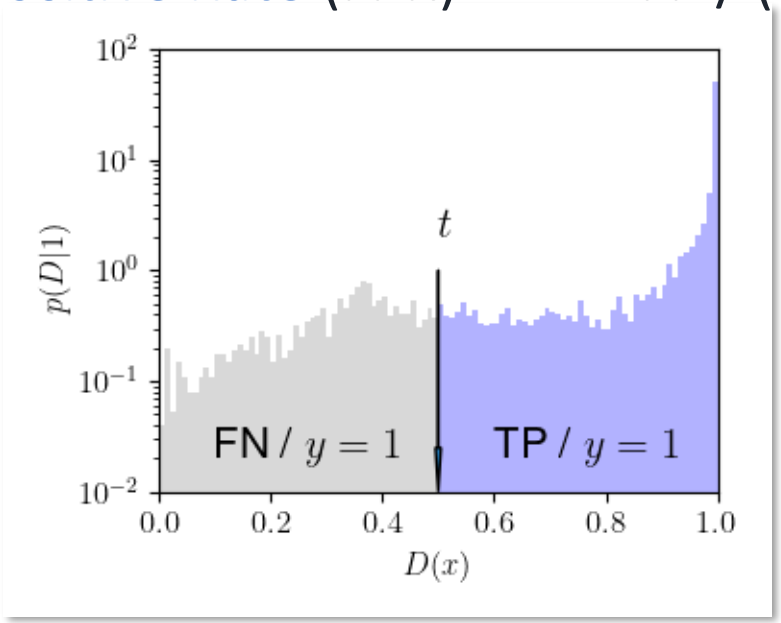
Accuracy = (TP + TN) / (TP + TN + FN + FP)

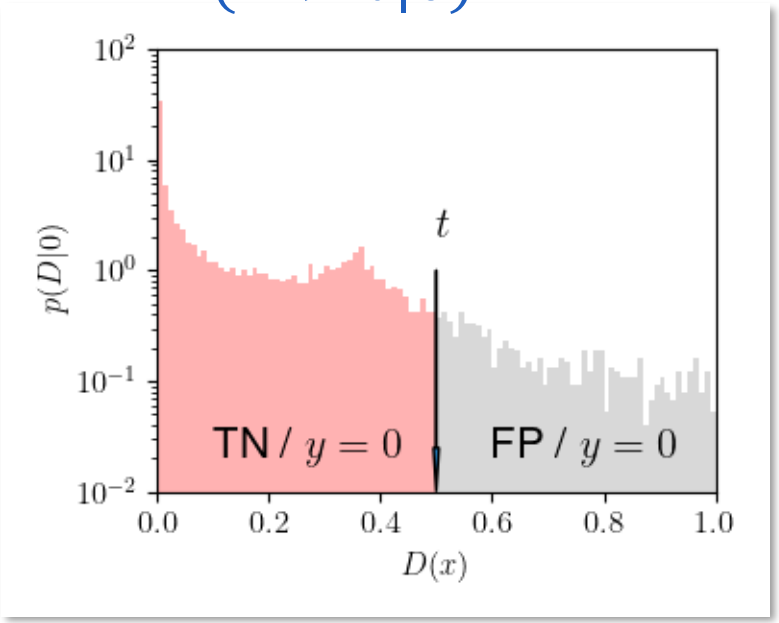
Precision = TP / (TP + FP) $P(y = 1|D > t)$

Recall = TP / (FN + TP)

True Positive Rate (TPR) = TP / (FN + TP) $P(D > t|1)$

False Positive Rate (FPR) = FP / (TN + FP) $P(D > t|0)$





ROC and AUC

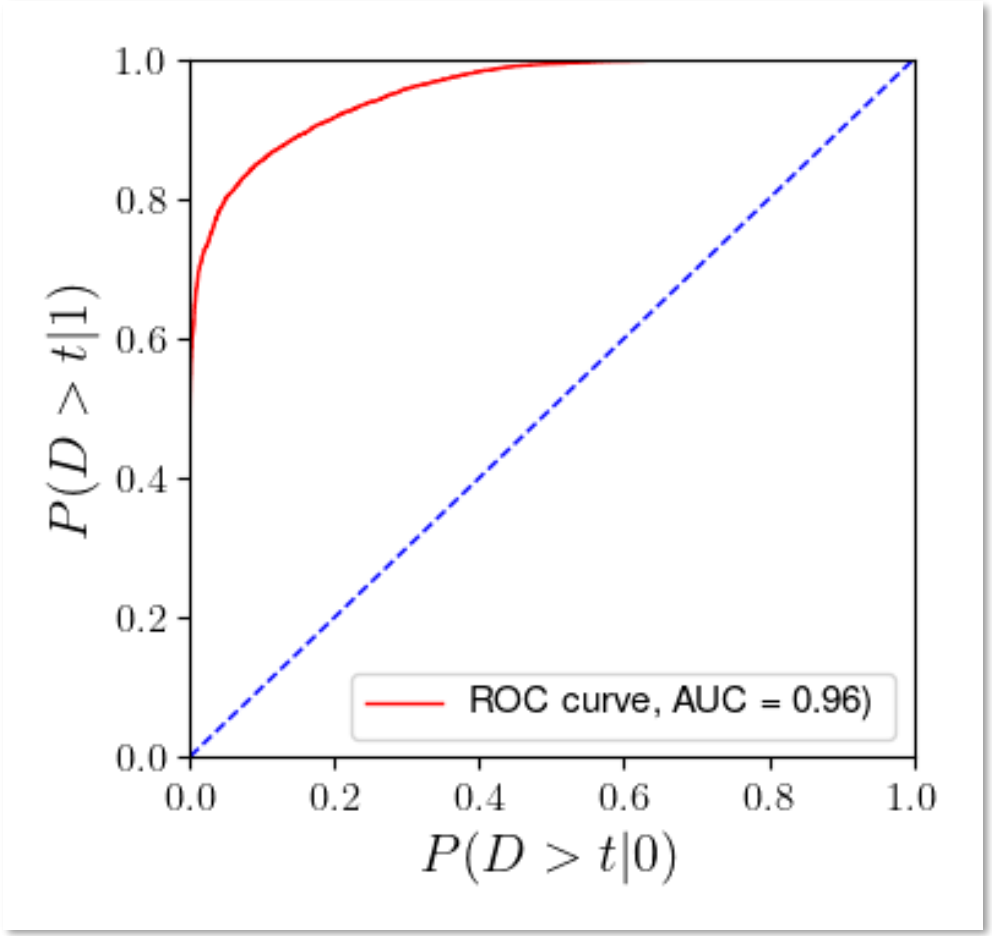
The Receiver Operating Characteristic (ROC) curve is a plot of TPR vs. FPR, that is,

$P(D > t|1)$ vs. $P(D > t|0)$

for different t, where (usually)

$$D(x) = \frac{p(x|1)}{p(x|1) + p(x|0)}$$

AUC: Area Under Curve



Summary

- The quadratic and binary cross entropy losses are the most widely used loss functions in machine learning.

- In addition to the ROC curve and the AUC, the performance of a classifier can be characterized with several other numbers including:
  - False Negatives (FN)
  - True Positives (TP)
  - True Negatives (TN)
  - False Positives (FP)

  which can be represented in a confusion matrix.

BEYOND CLASSIFICATION

Beyond Classification (1)

Notice that the discriminant

$$D(x) = \frac{p(x|1)}{p(x|1) + p(x|0)}$$

can be rewritten as

$$p(x|1) = \left[\frac{D(x)}{1 - D(x)}\right] p(x|0)$$

The above is often referred to as the **likelihood ratio trick**.

If $p(x|0)$ is a *known* density from which the $y = 0$ vectors are sampled, then the discriminant can be repurposed as a **probability density estimator**.

Beyond Classification (2)

Consider a dataset $\mathrm{U}[x]$, of size N, labeled $y = 1$. The goal is to approximate its associated probability density $u(x)$.

Algorithm

1. Generate another dataset $\mathrm{K}[x]$, of size N, labeled $y = 0$, where $x \sim k(x) \equiv p(x|0)$, a *known* density*.

2. Train a classifier $D(x)$ using a randomized mixture of the datasets $\mathrm{U}[x]$ and $\mathrm{K}[x]$ of sample size $2N$.
3. Approximate the unknown density $u(x)$ using

$$u(x) = k(x)\left[\frac{D(x)}{1 - D(x)}\right]$$

* The symbol "$\sim$" means "sampled from".

Beyond Classification (3)

But we can go further*. Suppose our dataset U of sample size *N* is from a simulation that depends on a set of parameters θ.

Let's sample these parameters from a *known* prior, $\pi(\theta)$. For each sampled point θ, x is sampled from the *unknown* density $u(x|\theta)$ using the simulator.

The dataset, labeled $y = 1$, comprises *N* pairs $\{(x, \theta)\}$ that constitute a **point cloud** representation of the joint density $u(x, \theta)$.

* Baldi, P., Cranmer, K., Faucett, T. *et al.*
Parameterized neural networks for high-energy physics. *Eur. Phys. J. C* **76**, 235 (2016).

Beyond Classification (4)

Algorithm

1. Generate a dataset K of size *N*, labeled $y = 0$, where θ is sampled from the *same* prior $\pi(\theta)$. For each θ, sample $x \sim k(x|\theta)$, where $k(x|\theta)$ is a *known* conditional density.

2. Train a classifier $f(x, \theta, \omega)$ with an admixture of U and K.

3. Compute $u(x, \theta) = k(x|\theta)\pi(\theta)\left[\frac{D(x,\theta)}{1-D(x,\theta)}\right]$.

4. Divide by $\pi(\theta)$:

$$u(x|\theta) = k(x|\theta)\left[\frac{D(x,\theta)}{1-D(x,\theta)}\right]$$

Beyond Classification (5)

The ratio

$$\frac{D(x,\theta)}{1-D(x,\theta)}$$

could have a large dynamic range.

If so, instead of approximating $D(x,\theta)$, it may be better to use the appropriate* loss function to approximate

$$f(x,\theta) = \log\left[\frac{D(x,\theta)}{1-D(x,\theta)}\right]$$

and then compute

$$\log u(x,\theta) = \log k(x \mid \theta) + \log \pi(\theta) + f(x,\theta)$$
$$\log u(x|\theta) = \log k(x \mid \theta) + f(x,\theta)$$

* Which one?